WHY SERVER-SIDE VALIDATION IS CRITICAL: UNDERSTANDING IDOR & OTP BYPASS RISKS

CONFIDENTIAL

Document Prepared

By

Muthu Kumar. A

About the Author

Muthu Kumar A is a senior Business Analyst and Quality Analyst with 10+ years of experience across Business Analysis, Software Testing, IT Infrastructure, and Operational Support.

Disclaimer

The concepts discussed in this article are based on general security principles and do not reference any specific organization or system. The intent is purely educational and aimed at promoting secure application design practices.

CONFIDENTIAL

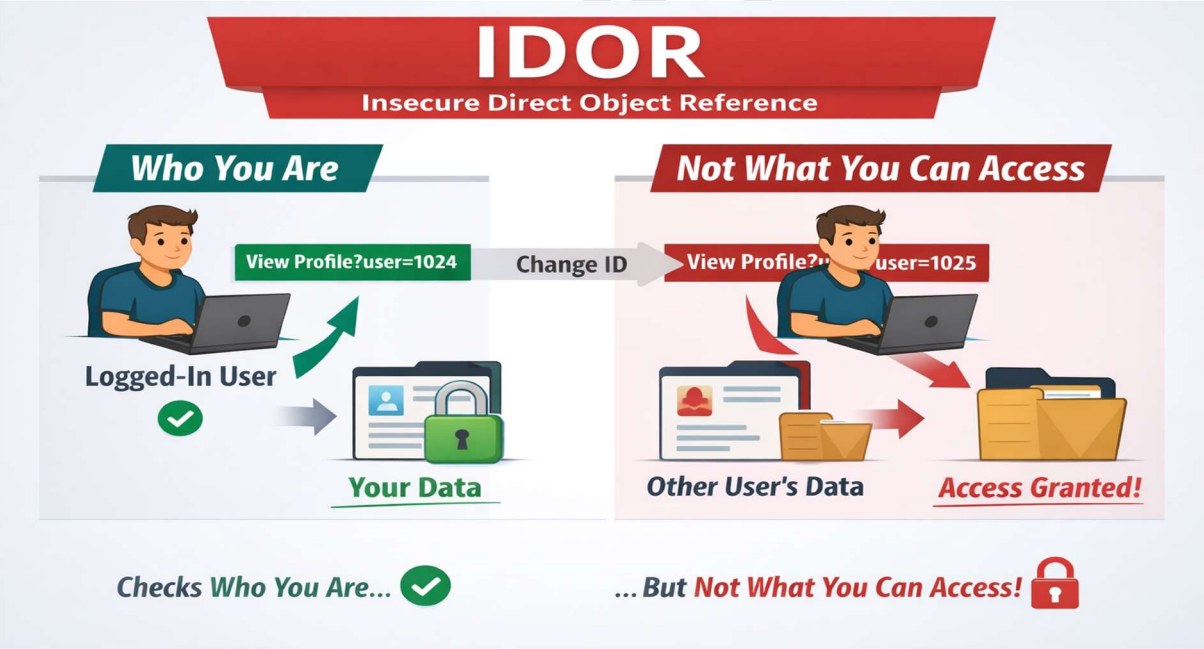
What is IDOR?

IDOR (Insecure Direct Object Reference) is a security vulnerability that occurs when an application exposes internal object identifiers (such as user IDs, account numbers, or document IDs) and fails to properly verify whether the authenticated user is authorized to access that specific resource.

This means:

- You are properly logged in.
- However, the application does not confirm that the data you are requesting actually belongs to you.
- When ownership validation is missing at the server level, sensitive data can become accessible to unintended users — which makes IDOR a serious access control vulnerability

Fig.1 IDOR Scenario Illustration.



From fig. 1, the user is successfully authenticated using valid credentials. However, the user attempts to access data that does not belong to them. In this scenario, authentication is working correctly — but authorization is missing.

The system verifies the user's identity, yet it fails to verify whether the user is permitted to access the requested resource.

This represents a classic case of:

- Authentication without Authorization
- Identity Verified, Permission Not Verified
- Checks Login, Not Ownership

The heading **"Not What You Can Access"** emphasizes the core issue:

- Missing ownership validation
- Broken access control logic
- Lack of server-side authorization enforcement

In secure application design, authentication and authorization must work together. Verifying identity alone is not sufficient — the system must also enforce strict resource ownership and access control validation at the server level.

How Does an Application Unintentionally Expose IDOR?

IDOR vulnerabilities typically arise when applications expose internal object identifiers—such as user IDs, document IDs, or account numbers—through request parameters, URLs, cookies, or browser storage, without enforcing proper server-side authorization checks.

When Client-Side Trust Becomes a Security Threat.

Because client-side elements such as request parameters and cookies can be modified, any missing server-side access control logic can unintentionally make sensitive data accessible. This is why Broken Access Control, including IDOR, remains a top security risk as highlighted in the OWASP Top 10 under Broken Access Control.

How to Prevent IDOR in Applications

Preventing Insecure Direct Object Reference (IDOR) is primarily about enforcing strong server-side authorization controls.

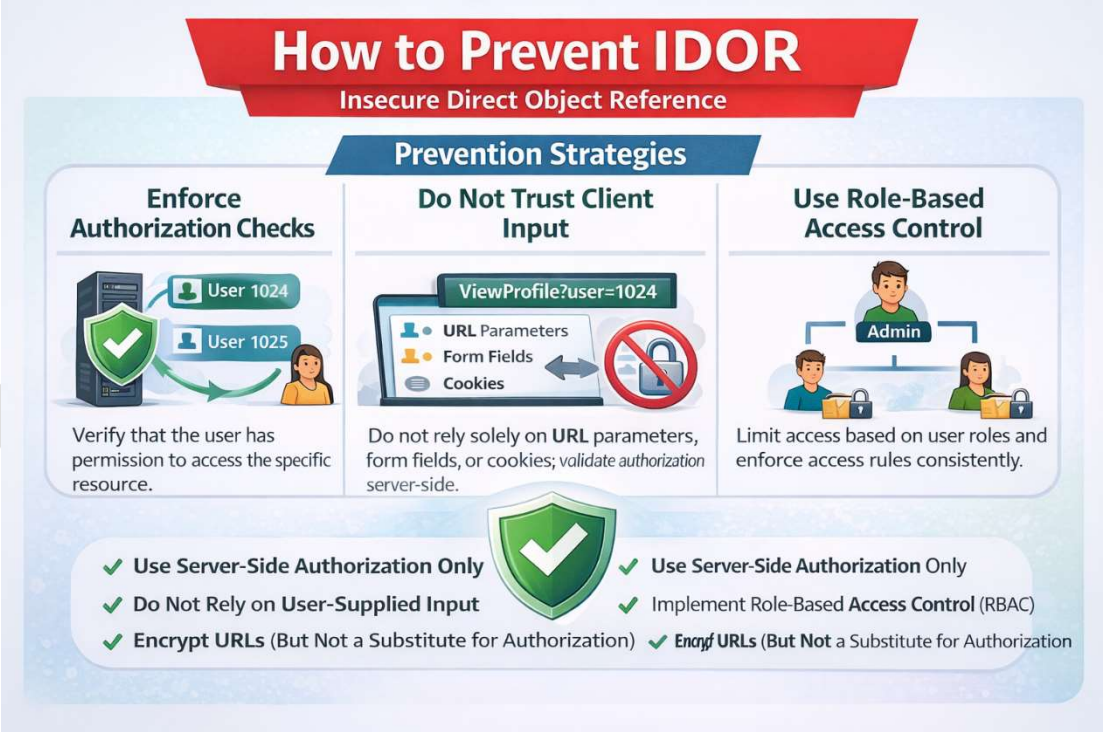
- Enforce Server-Side Authorization Checks. If the user is accessing more sensitive information or the application screen is handling more sensitive information, then

CONFIDENTIAL

Authentication alone is not sufficient. The system must verify that the authenticated user is authorized to access the requested resource.

- Do Not Trust Client-Controlled Input. - URLs and request parameters may contain identifiers required for application functionality. However, since these values are controlled by the client, they must never be trusted without server-side validation. Mostly data are to be exposed via,
    - URL parameters
    - Request body identifiers
    - Hidden form fields
    - Cookies

Encrypting or obfuscating identifiers in URLs may reduce visibility of sensitive information. However, encryption alone does **not** prevent IDOR. If proper authorization checks are missing, encrypted values can still be reused or replayed.



What is OTP Authentication?

One-Time Password (OTP) is a secondary authentication mechanism used to verify user identity during login or sensitive operations (such as downloading confidential data). OTP strengthens authentication, but it does not replace authorization controls. It is designed to strengthen authentication by adding a second verification factor.

OTP is typically:

- Time-bound
- Single-use
- Sent via SMS / Email / Authenticator apps

Where Should OTP Be Stored and How Is It Validated ?

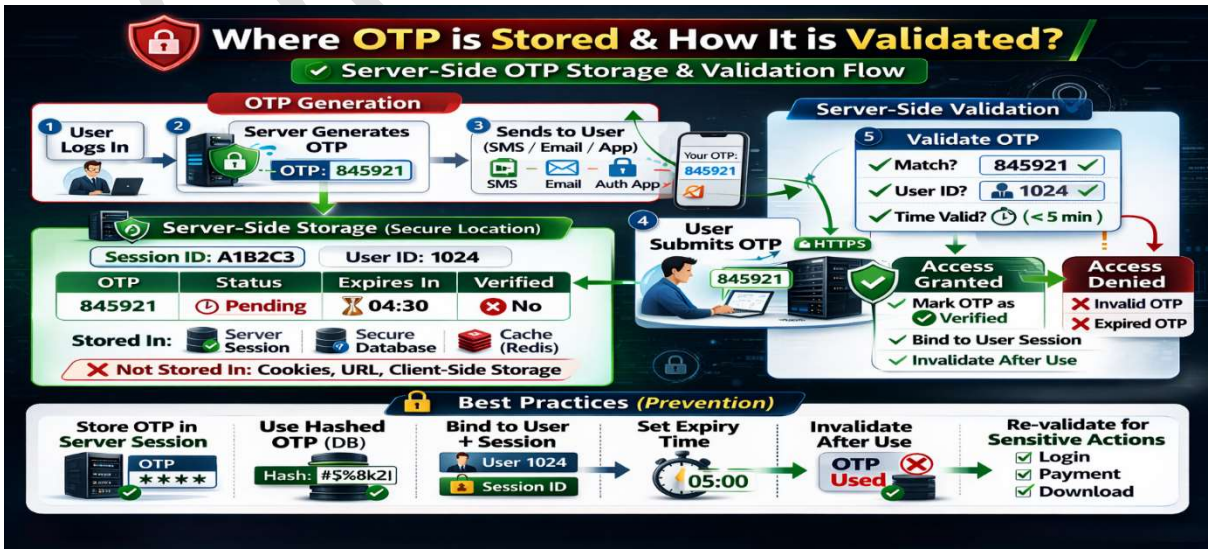
After OTP is generated:

- It should be stored in the **server session**
- Mapped to the authenticated user
- With a timestamp (expiry time)

When the user submits the OTP:

- The server compares it with the stored value
- If valid → mark as verified
- Then immediately invalidate it

This is the most secure approach.



CONFIDENTIAL

What will be happen when validating OTP in client side

If OTP (One-Time Password) validation is done only on the client side, it creates a serious security vulnerability.
Since client-side scripts can be manipulated, any OTP validation logic that relies solely on browser-side checks can be bypassed if not revalidated by the server. Therefore, OTP verification must always be enforced and validated at the server level.

When IDOR and OTP Bypass Exist in the Same Application…

Two security concerns stood out:

- IDOR (Insecure Direct Object Reference) – where the system validates who the user is, but fails to verify whether they are authorized to access specific data.
- OTP Bypass Risk – where OTP verification is implemented, but not consistently enforced at the server level for sensitive operations.

Individually, these may appear as separate defects. However, when viewed together, they often point to a broader issue:

- A structural gap in server-side access control enforcement.
- In many applications, authentication is handled properly. But authentication alone is not sufficient.

Root Cause: Trust Boundary Failure

When applications assume that client-side controls are sufficient, security enforcement becomes inconsistent and vulnerable to manipulation.

Conclusion:

IDOR and OTP bypass may appear to be independent vulnerabilities, but both stem from the same fundamental weakness — inadequate server-side authorization enforcement. Authentication mechanisms alone cannot guarantee data protection if ownership and permission checks are not consistently validated at the server level.

True application security is achieved not by adding more controls, but by enforcing them at the correct trust boundary. Secure system design begins where client-side assumptions end.

*****End of the document*****

CONFIDENTIAL